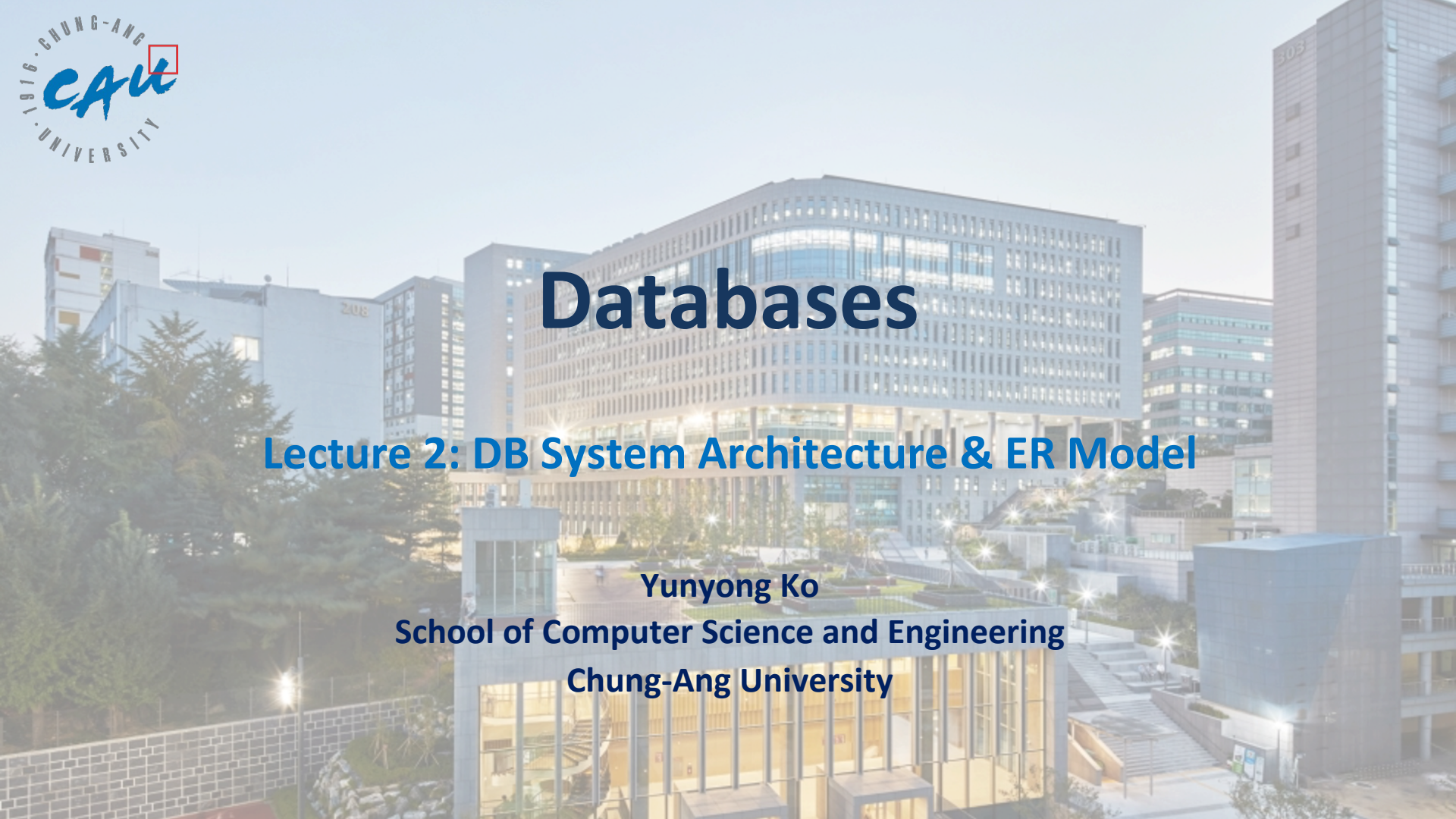


Databases

Lecture 2: DB System Architecture & ER Model

Yunyong Ko
School of Computer Science and Engineering
Chung-Ang University

Outline

❑ Database System Architecture

- Conceptual, representational, and physical data models
- Three-schema architecture and data independence



❑ ER Model: Entities and Attributes

- Process of conceptual data modeling
- Entity types and attributes
- Initial conceptual design

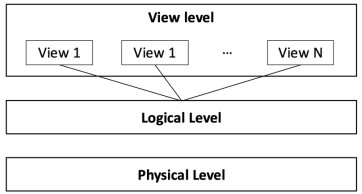
❑ ER Model: Relationships and Constraints

- Relationship types, constraints, and attributes
- Weak entity types
- Refining conceptual design

Data Model and Abstraction

What is a Data Model?

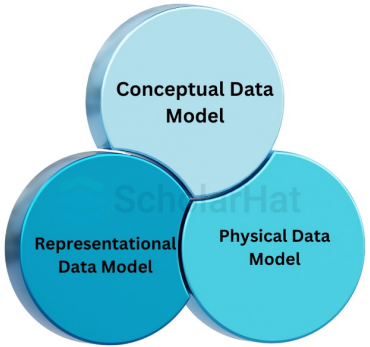
- A collection of concepts that ***describe the structure of a database***
- Providing the necessary "**Tools**" to achieve *data abstraction*
  - e.g., data types, relations, and constraints that apply to the data
  - e.g., basic operations for specifying retrievals and updates on the DB



The highlighting of the essential features for understanding of data
The suppression of details of data organization and storage

Three Categories of Data Models

- Conceptual data models (high-level)
- Representational data models (between the two levels)
- Physical data models (low-level)



Conceptual Data Model

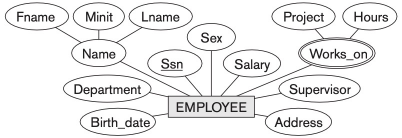
❑ Providing Concepts Close to *the Way Users Perceive Data*

- Example: Company Database

Employee



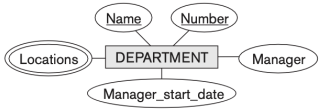
Name, Salary, Age,
Address, Department, …



Department



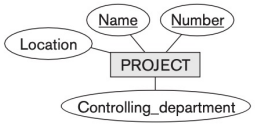
Name, Dnumber,
Manager, Location, …



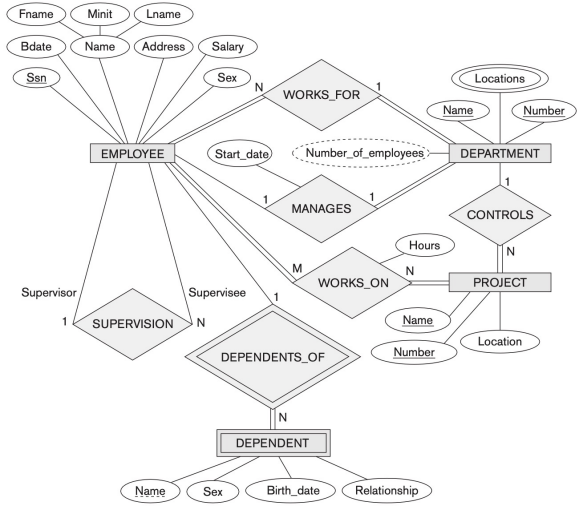
Project



Name, Pnumber,
Controlling_Dept., Location, …



Careful requirement analysis

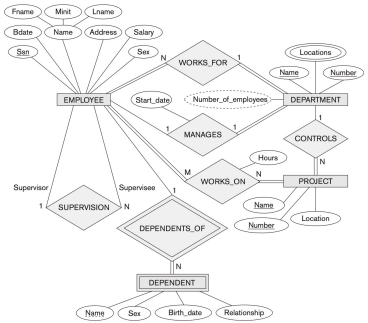


Conceptual Design for Company Database

Representational Data Model

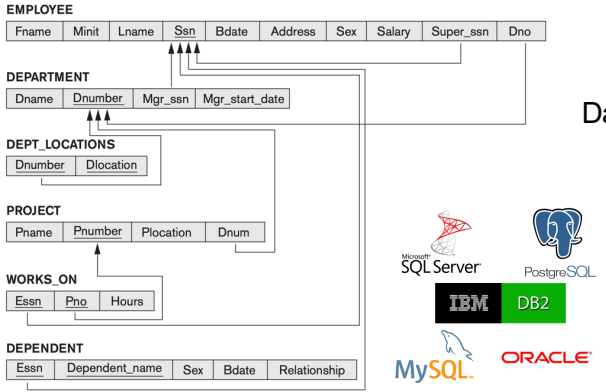
❑ Describing *How Data is Managed by Software*

- Managed by Database management system (DBMS)
- Bridging the Gap between Conceptual and Physical Models



Conceptual data design

Data definition



Hiding many details of physical data storage on disk,
but can be implemented on a computer system directly

Data manipulation



Physical Data Model

❑ Describing *How Data is Stored in a Computer System*

- Representing information such as *record formats*, *orderings*, and *access paths*
  - ❑ The provided concepts are meant for computer specialists, not for end users

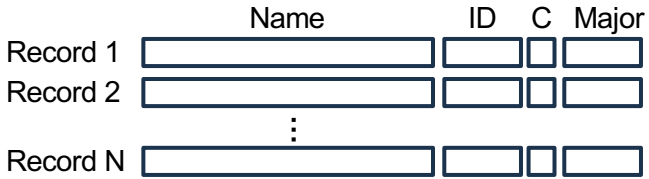


STUDENT

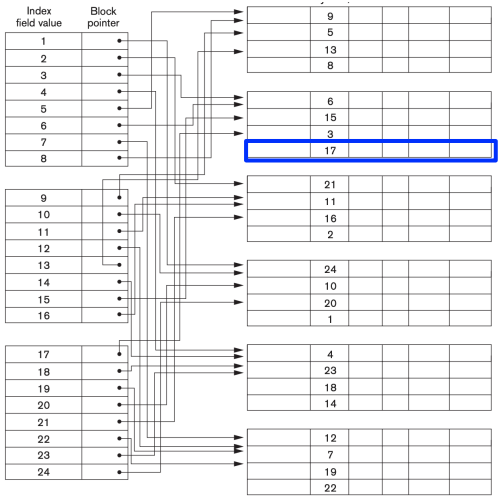
Name	Student_number	Class	Major



Data Item Name	Starting Position in Record	Length in Characters (bytes)
Name	1	30
Student_number	31	4
Class	35	1
Major	36	4



How to access a specific data?

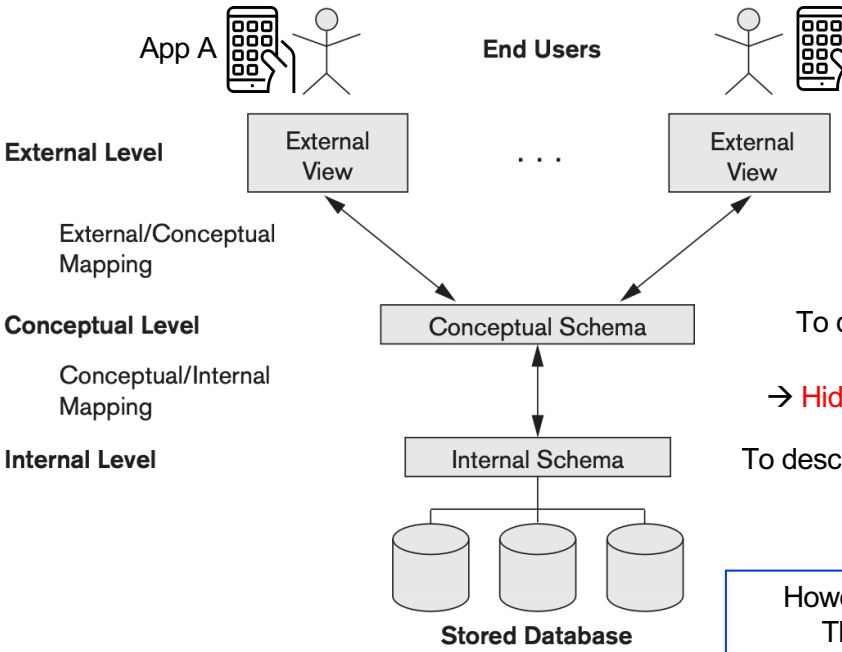




Three-Schema Architecture

❑ Goal: *To Separate User Apps from Physical DB*

- For the independence between a user program and data



To describes the part of the database structure that a particular user group is interested in

To describes the logical structure of the databases using a **representational data model**

→ Hiding the details of the physical storage structure

To describes the physical storage structure of the database using a **physical data model**

However, the actual data is stored at the physical level only.
Thus, how can the DBMS support each user request?

Three-Schema Architecture: Mapping

❑ Transforming Requests and Results between Levels

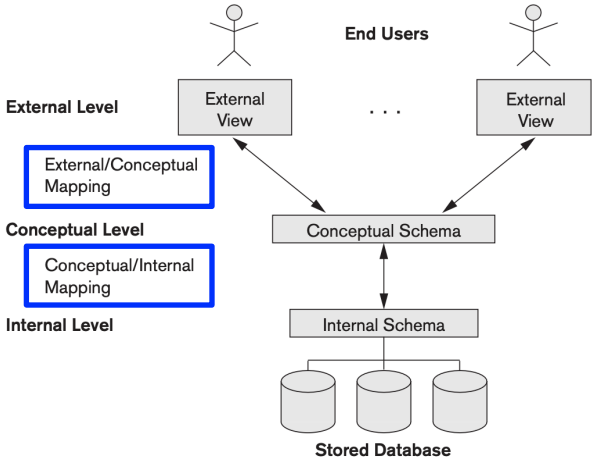
- Actual data is stored at the physical level only
- The DBMS must transform a request specified on a high-level schema into a request against a lower-level schema

Transform a user request:

External schema → Conceptual schema

Conceptual schema → Internal schema

Processing the request
(e.g., data retrieval)



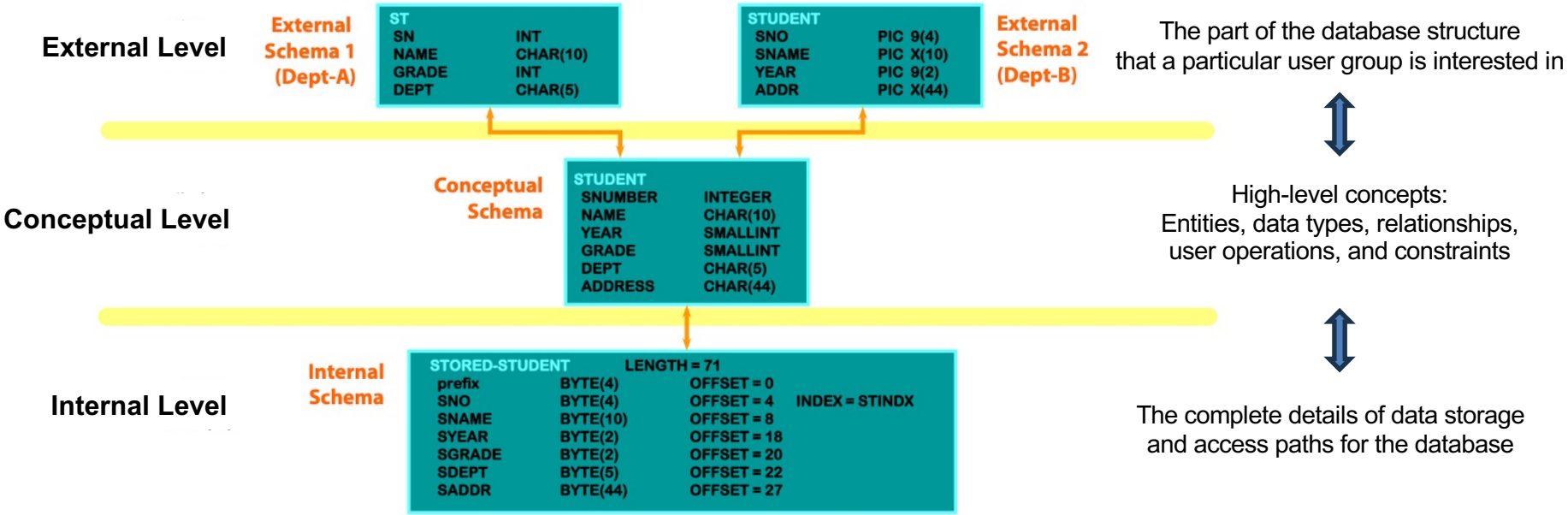
Reformatted to match the user's external view

The data is extracted from the stored database

Mapping information is also stored in the catalog

Three-Schema Architecture: Example

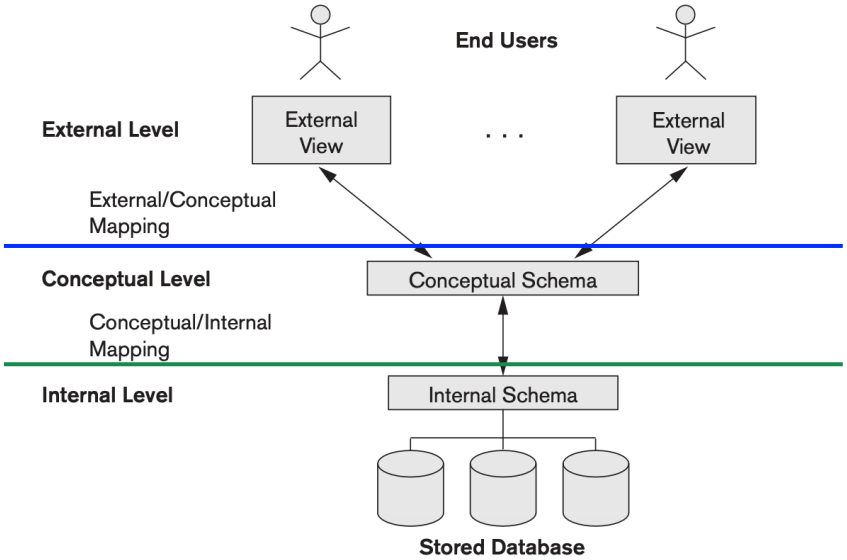
❑ University Database



Data Independence

❑ What is the Data Independence?

- *The capacity to change the schema at one level of a database system without having to change the schema at the next higher level*



What if we change the conceptual schema?
E.g., adding a new data or removing a data

Logical Data Independence

→ The Capacity to Change the Conceptual Schema

Physical Data Independence

→ The Capacity to Change the Internal Schema

What if we change the physical storage organization?
(e.g., providing an access path to improve retrieval speed)

Significantly reduces the maintenance overhead of existing application programs

Outline

- Database System Architecture
  - Conceptual, representational, and physical data models
  - Three-schema architecture and data independence
- **ER Model: Entities and Attributes**
  - Process of conceptual data modeling
  - Entity types and attributes
  - Initial conceptual design



- **ER Model: Relationships and Constraints**
  - Relationship types, constraints, and attributes
  - Weak entity types
  - Refining conceptual design

Database Design Process

❑ Phase 1: Requirements Collection and Analysis

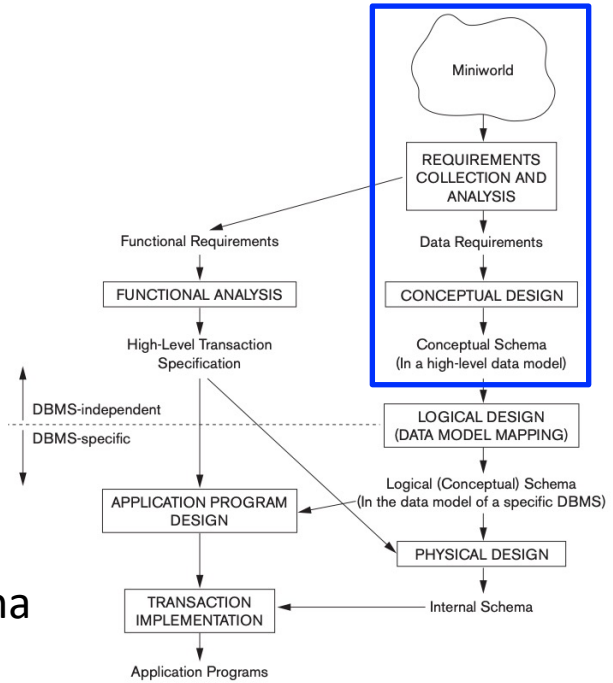
- Characterize the data needs of the prospective DB users
- Collect and analyze data requirements for the database
  - **Output**: a set of users' detailed requirements

❑ Phase 2: Conceptual Design

- Choosing a data model
- Translating the requirements into a conceptual schema

❑ Phase 3: Implementation

- Implementing the DB based on the conceptual DB schema
  - Logical design: the database schema
  - Physical design: the physical layout of the DB



Requirement Analysis

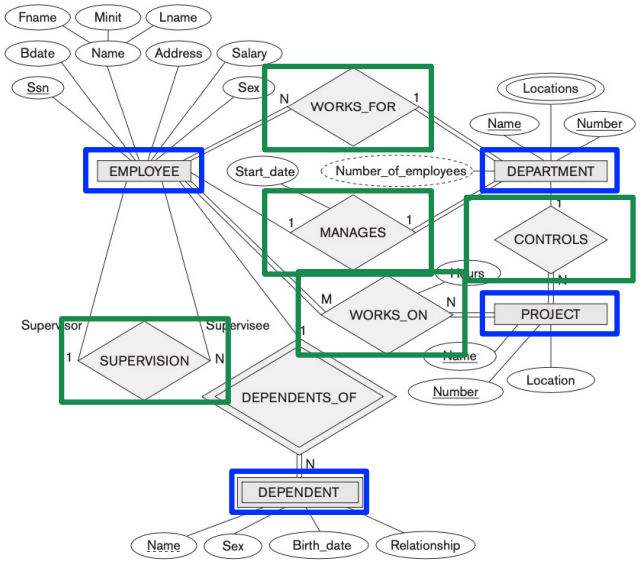
What Information Should be Managed?

- We want to keep track of a company's employees, departments, and projects
- **Department**
  - The company is organized into '**department**'s
    - Each has a unique name, a unique number, locations, manager, start date when that manager began, …
- **Project**
  - A department can control a number of '**project**'s
    - Each has a unique name, a unique number, a single location, controlling department
- **Employee**
  - '**Employee**'s are working in the company
    - Each has a name, SSN, address, salary, gender, birth date, projects that the employee works on, …
- **Dependent**
  - For insurance purposes, the '**dependent**'s of each employee are needed
    - Each has a first name, gender, birth date, relationships to the employee, …

Conceptual Data Modeling

❑ Entity-Relationship (ER) Model

- Describing data as a collection of ***entities*** and ***relationships***
  - **Entity**: a “thing” or “object” that is distinguishable from others
  - **Relationship**: an association among several entities
- Represented diagrammatically by an ***ER diagram***



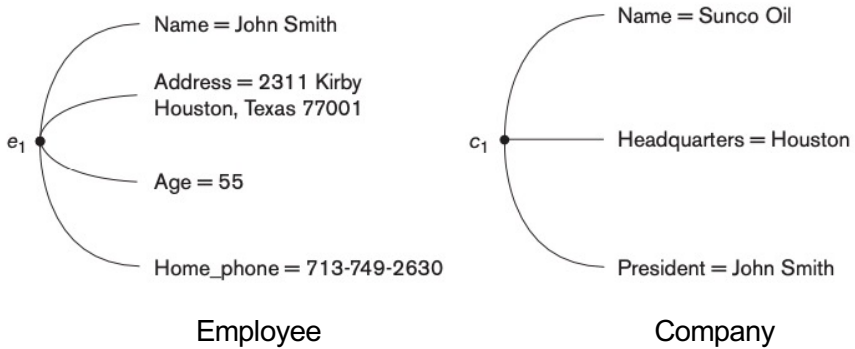
Entities and Attributes

Entity

- *A thing* or *an object* in real world with independent existence
  - An entity may be an object with a physical existence or a conceptual existence
    - E.g., employee vs company

Attribute

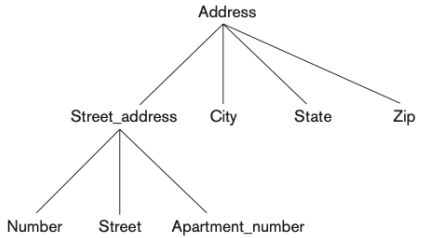
- A particular *property* that describes the entity
  - E.g., employee: Name, address, age, and home_phone
  - E.g., company: Name, headquarters, and president



Categorization of Attributes

Category 1: Composite vs Simple Attributes

- **Composite** attributes: *can be divided* into smaller subparts
  - e.g., Address consists of Street_address, city, state, and zip
- **Simple** (atomic) attributes: *non-divisible*
  - e.g., city, state, zip, number, street, apartment_number



Category 2: Single-Valued vs Multivalued Attributes

- **Single-valued** attributes: have a *single value* for a particular entity
  - e.g., (most attributes) age, gender, birth date, SSN, …
- **Multivalued-valued** attributes: have *a set of values* for the same entity
  - e.g., phone numbers, college degrees, …

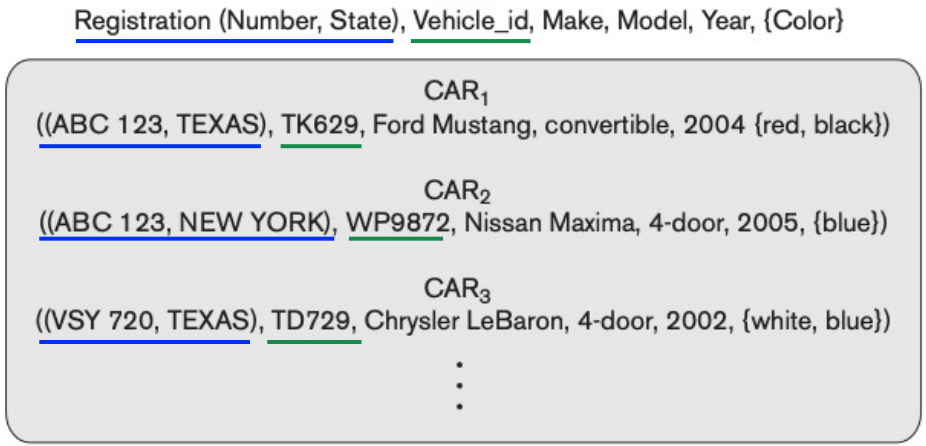
Category 3: Stored vs Derived Attributes

- **Stored** attributes: actually *stored* in the database
  - e.g., birth date
- **Derived** attributes: not stored in the database, but *can be derived* from others
  - e.g., age (derived from the birth date)

Key Attributes of an Entity Type

What is a Key Attribute?

- An attribute whose value is ***distinct*** for each individual entity
  - *i.e., used to identify each entity uniquely*
  - e.g., student number of STUDENT entity and SSN of PERSON entity
- Multiple attributes can form a key
- An entity type can have more than two keys

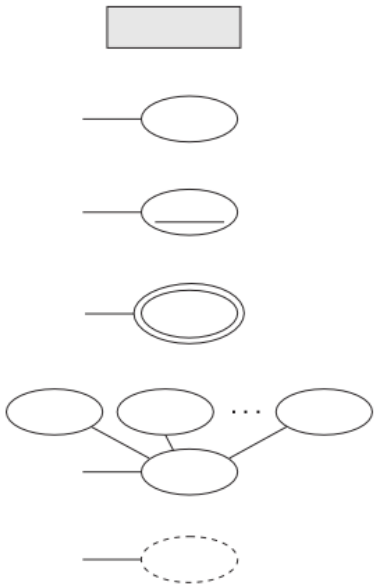


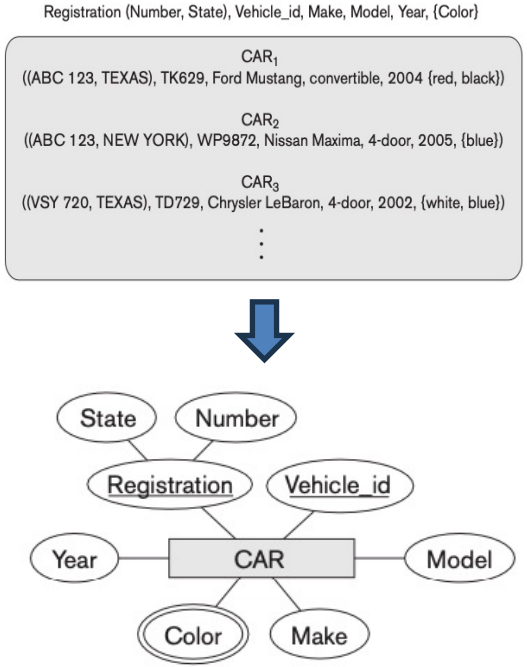
The "Registration" attribute consists of two attributes

The CAR entity has two keys: Registration and Vehicle_id

Representation of Entity Types

❑ How to Represent an Entity Type and its Attributes?





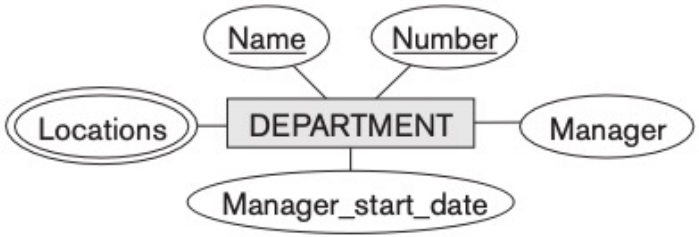
Initial Conceptual Design: COMPANY

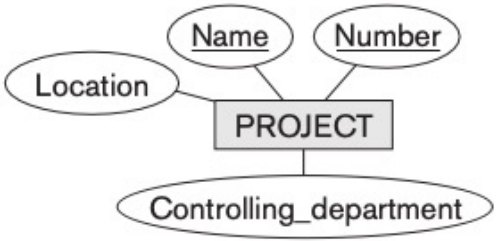
❑ DEPARTMENT

- **Attributes**: Name, Number, Locations, Manager, and Manager_start_date
  - ❑ Name and Number are key attributes (underlined) since each was specified to be unique
  - ❑ Locations is the multivalued attributed

❑ PROJECT

- **Attributes**: Name, Number, Location, Controlling_department
  - ❑ NAME and Number are key attributes

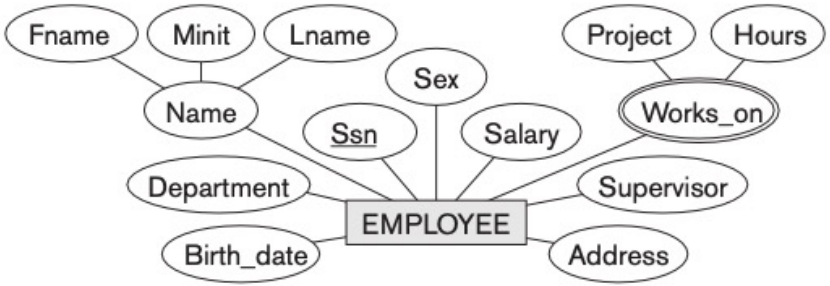


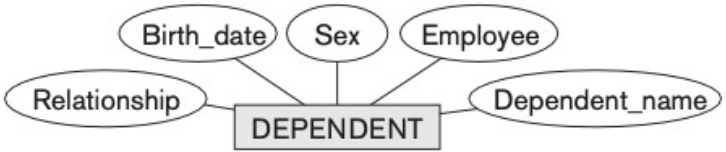


Initial Conceptual Design: COMPANY (Cont'd)

❑ EMPLOYEE

- **Attributes:** Name, SSN, Address, Salary, Birth_date, Sex, Department, Supervisor, Works_on
  - ❑ SSN is a key attribute , Name is a composite attribute (Fname, Minit, Lname)
  - ❑ An employee can work on several projects,
    - The number of hours per week an employee works on each project is managed
- **DEPENDENT**
  - ❑ **Attributes:** Employee, Depedent_name, Sex, Birth_date, Relationship

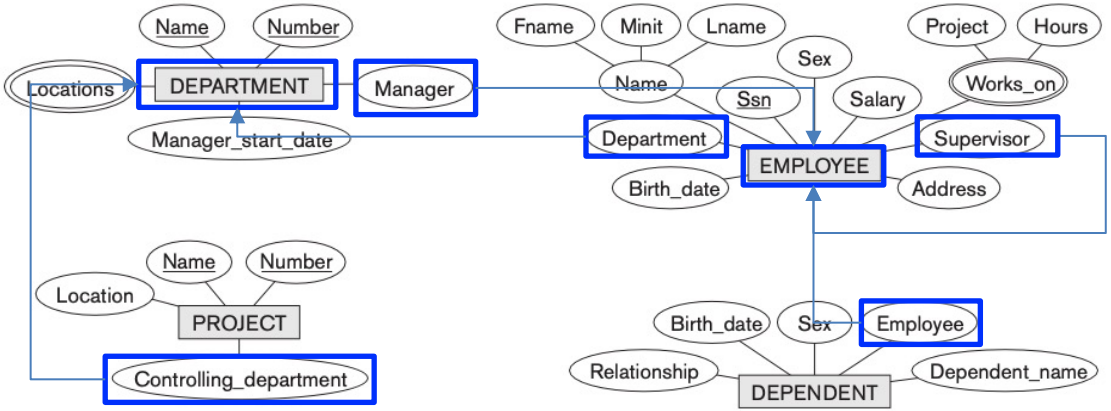




Initial Conceptual Design: COMPANY (Cont'd)

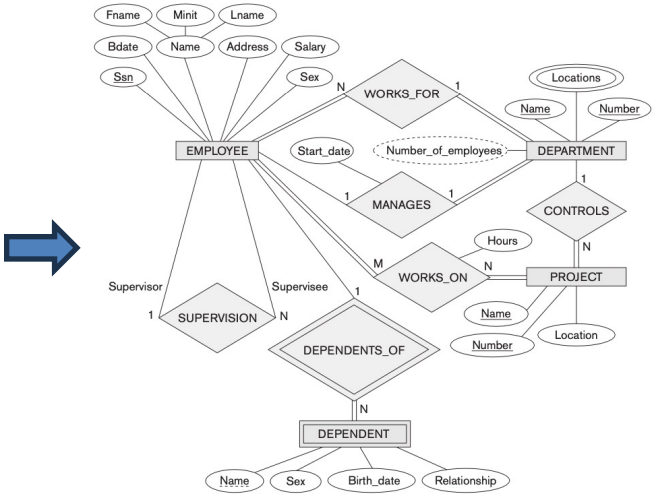
❑ Is It Enough to Model Relationships Among Entities?

Q. What relationships do they have?



(Implicit) Relationships are captured in the form of attributes

→ These references should be represented as "Relationships"



Let's represent relationships among entities explicitly

Outline

- Database System Architecture
  - Conceptual, representational, and physical data models
  - Three-schema architecture and data independence
- ER Model: Entities and Attributes
  - Process of conceptual data modeling
  - Entity types and attributes
  - Initial conceptual design
- **ER Model: Relationships and Constraints**
  - Relationship types, constraints, and attributes
  - Weak entity types
  - Refining conceptual design



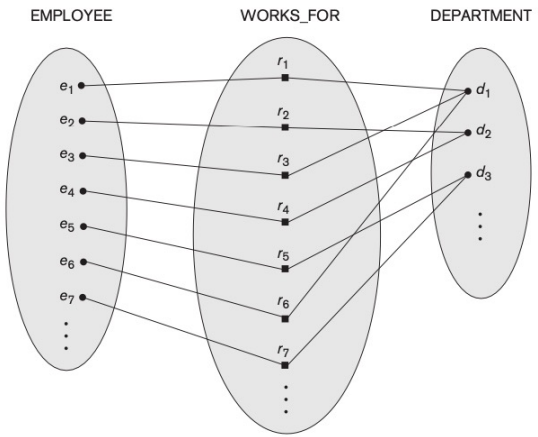
Relationship Type

Relationship Type

- Described by its name and participating entity types
- Define a set of associations among *participating entity types*

Example: WORKS_FOR

- Relationships between EMPLOYEE and DEPARTMENT
  - Associating each employee with the department for which the employee works



Participating entity types:
EMPLOYEE and DEPARTMENT

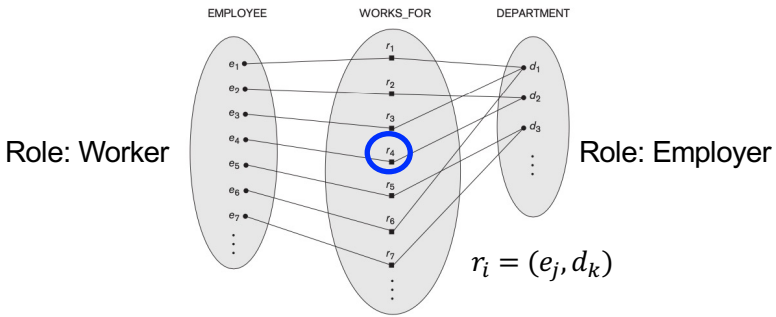
Relationship: Degree and Role Name

❑ Degree of a Relationship Type

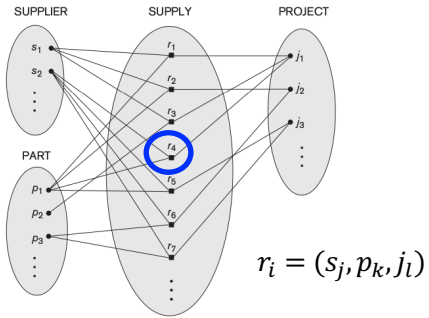
- Number of participating entity types (binary, ternary, …)
- Relationships can generally be of any degree
  - ❑ Most relationships in a DB system are binary

❑ Role Name of a Relationship Type

- Signifying a role that a participating entity plays in each relationship instance
  - ❑ It helps us to understand what the relationship means



Binary Relationship

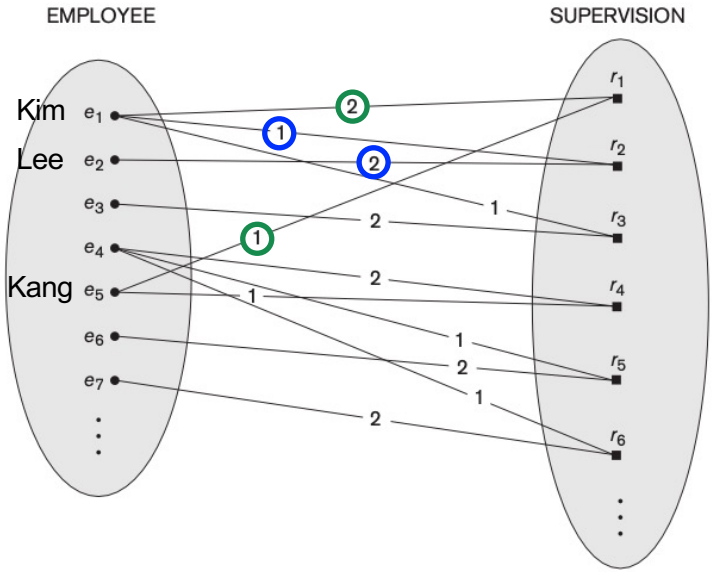


Ternary Relationship

Relationship: Recursive Relationships

❑ Recursive Relationship

- The same entity type participates more than once in a relationship type
  - ❑ Essential to specify the role name for distinguishing the participating entities



Line marked '1': the supervisor role

Line marked '2': the supervisee role

$r_2 = (e_1, e_2)$
"Kim" is the supervisor of "Lee"

$r_1 = (e_5, e_1)$
"Kim" is a supervisee of "Kang"

Constrains on Relationship Types

❑ Why Constraints of a Relationship Type Need?

- Relationship types usually have certain constraints about participating entities
  - e.g., *each employee must "**work for" exactly one department***
  - e.g., *each employee can **"work on" multiple projects** and should be "**supervised" by exactly one employee***

❑ Types of Constraints

- Constraint 1: Cardinality Ratio
  - To specify **the maximum number of relationship instances that an entity can participate in**
  - Three types of cardinality ratios for a binary relationship: 1:1, 1:N, M:N
- Constraint 2: Participation Constraint
  - To specify **whether the existence of an entity depends on its being related to another entity**
  - Two types of a participation constraint
    - **Total**: every entity should participate in the relationship
    - **Partial**: every entity does not need to participate in the relationship

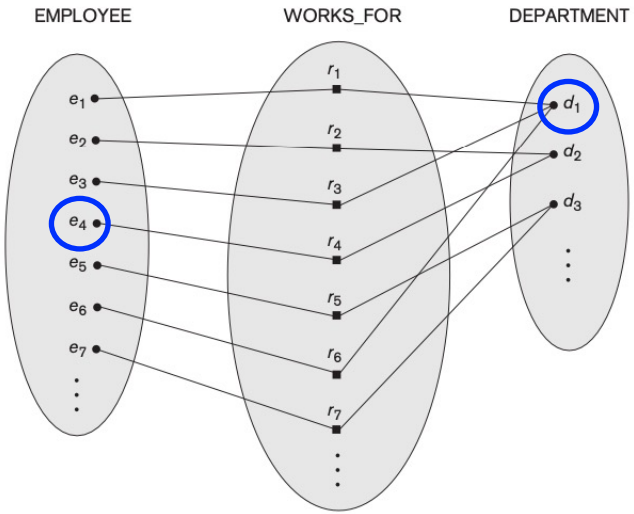
Constraint 1: Cardinality Ratio

❑ 1:N Cardinality Ratio

- A 1:N binary relationship relates an entity to any number of entities

N-side entity **1-side entity**

An employee can be related to at most one department



Each department can be related to any number of employees

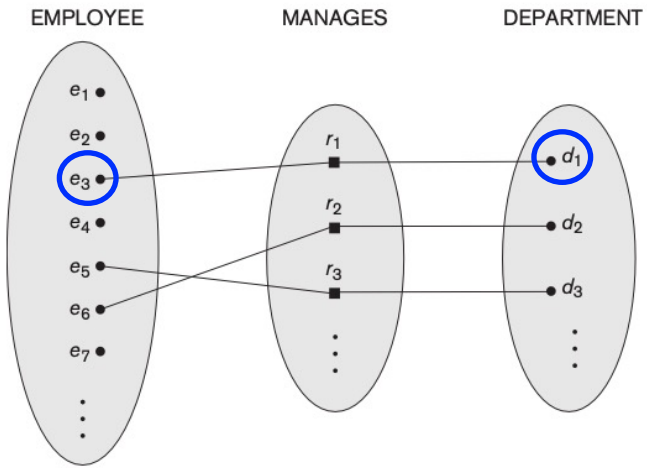
Real-world Constraints:
(1) An employee can work for only one department
(2) A department can employ any number of employees

Constraint 1: Cardinality Ratio (Cont.)

❑ 1:1 Cardinality Ratio

- A 1:1 binary relationship relates an entity to another entity

An employee can be related to at most one department



Each department can be related to an employee as a manager

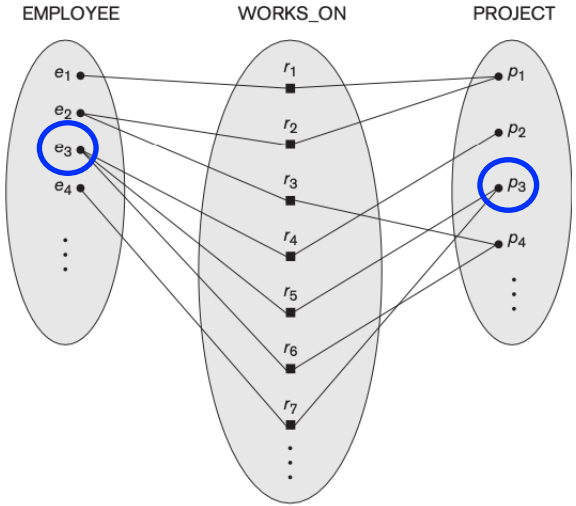
Real-world Constraints:
(1) An employee can manage at most one department
(2) A department can have at most one manager

Constraint 1: Cardinality Ratio (Cont.)

❑ M:N Cardinality Ratio

- A M:N binary relationship relates any number of entities to any number of entities

Each employee can be related to any number of projects

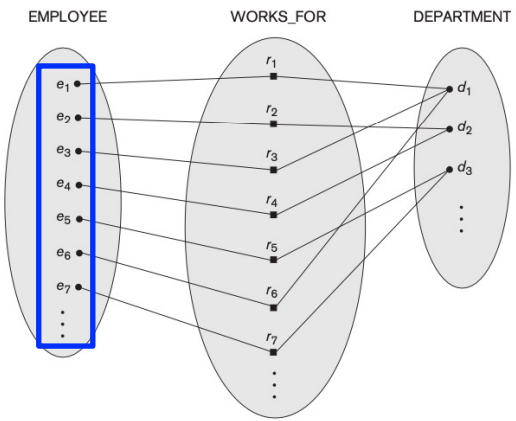


Each project can be related to any number of employees

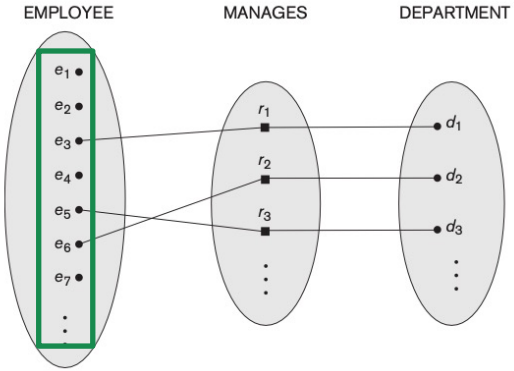
Real-world Constraints:
(1) An employee can work on any number of projects
(2) A project can be conducted by any number of employees

Constraint 2: Participation Constraint

- **Total Participation**
  - Every entity should participate in the relationship
- **Partial Participation**
  - Every entity does not need to participate in the relationship



The participation of EMPLOYEE in WORKS_FOR:
Total Participation

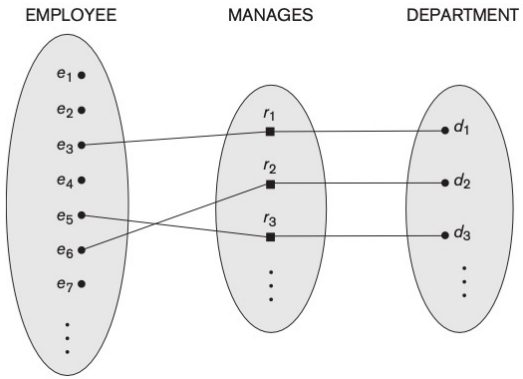


The participation of EMPLOYEE in MANAGES:
Partial Participation

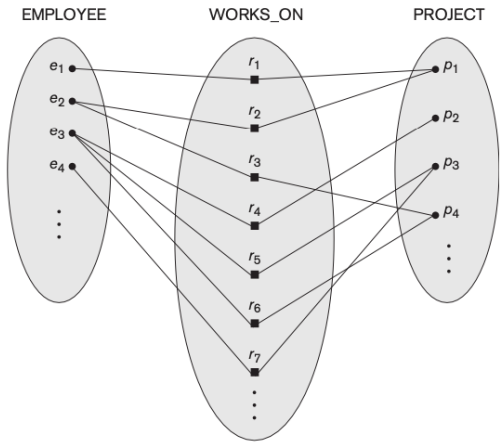
Attributes of Relationship Types

❑ Relationship Types Can Have Attributes

The company wants to keep track of:
(1) the **start date** of a manager when the employee manage the department
(2) the **number of working hours** of each employee on a specific project and



StartDate for MANAGES

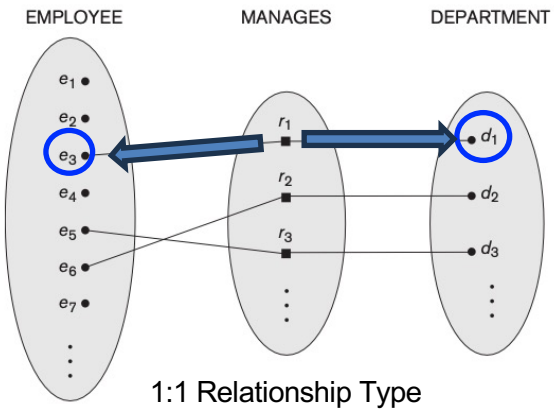


Hours for WORKS_ON

Attributes of Relationship Types (Cont.)

❑ Representation of 1:1 and 1:N Relationship Type Attributes

- The attributes can be migrated to one of the participating entity types

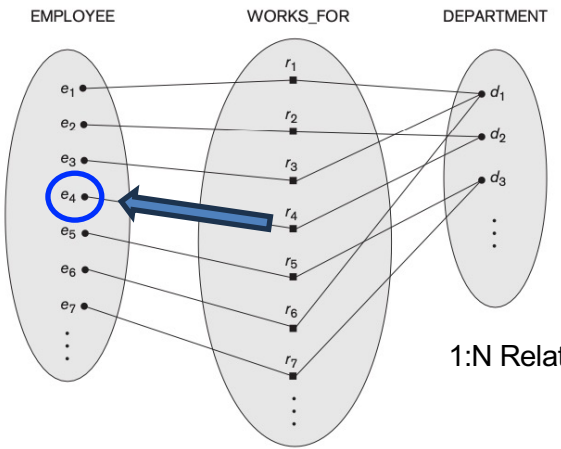


1:1 Relationship Type

StartDate for MANAGES

Migrated to **one of any relationship types**

Q. Which one is better?



1:N Relationship Type

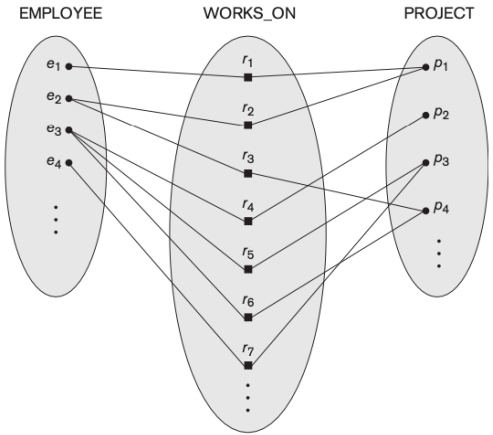
StartDate for WORKS_FOR

Migrated only to **the N-side of the relationship type**

Attributes of Relationship Types (Cont.)

❑ Representation of M:N Relationship Type Attributes

- Some attributes may be determined only by the *combination of participating entities* in a relationship instance, *not by any single entity*
  - *i.e., Those attributes cannot be migrated to any participating entities*



Hours for WORKS_ON

The number of hours per week an employee 'works on' a project is determined by an employee-project combination.

Hours cannot be migrated into one of the participating entities

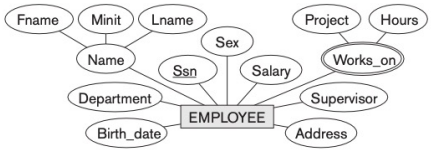
Weak Entity Types

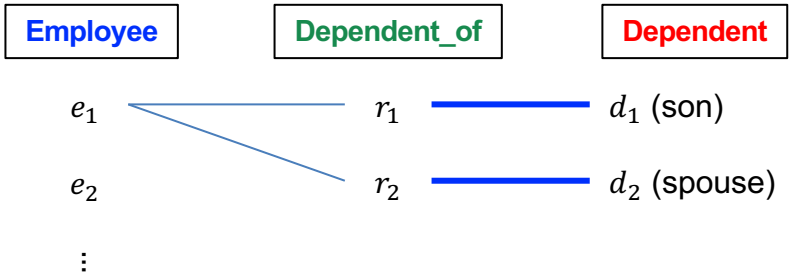
What is a Weak Entity Type?

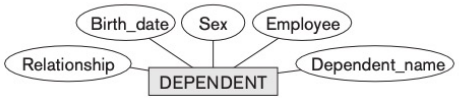
- An entity type has no key attributes of its own inside the entity type
- A weak entity type depends on its *identifying entity type*
  - Each weak entity must be associated with an identifying entity

Identifying Relationship

- Relating a weak entity type to its owner entity type
- A weak entity type always has a *total participation constraint*



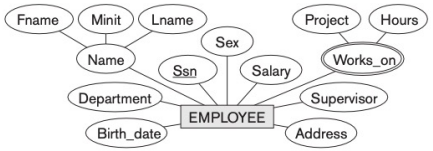


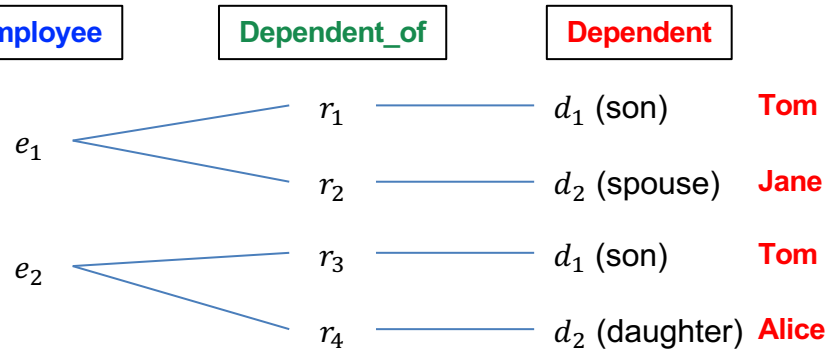


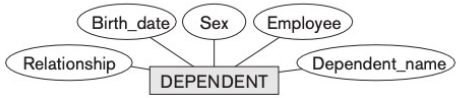
Weak Entity Types (Cont.)

❑ Partial Key

- To uniquely identify weak entities that are related to the same owner entity
- Example 1: EMPLOYEE-DEPENDENT
  - The **Dependent_Name** of DEPENDENT is the partial key since no two dependents have the same name



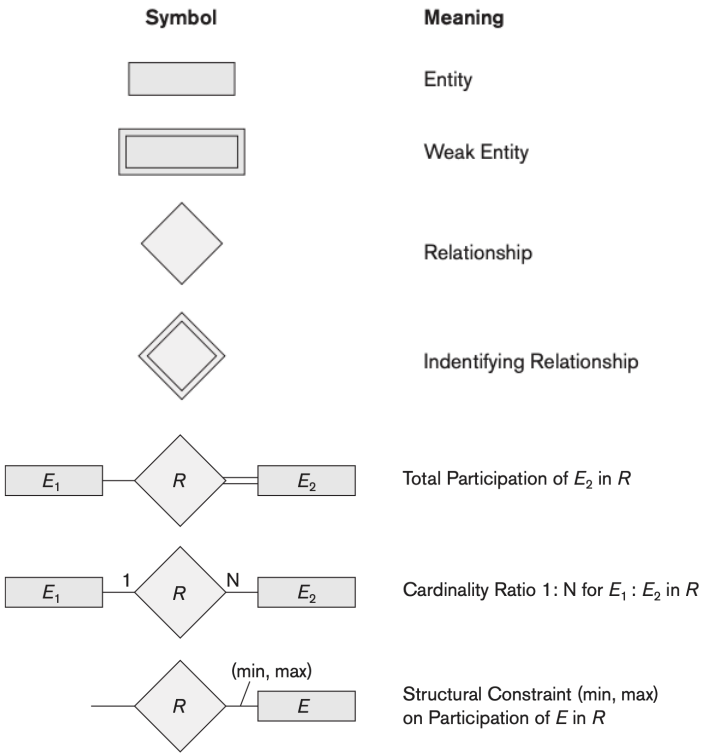


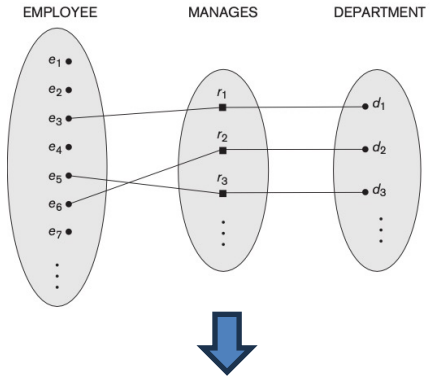


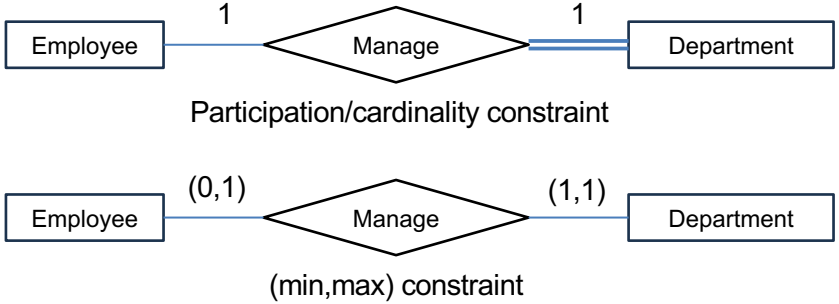
However, a partial key alone cannot identify its corresponding weak entity
Thus, it can be used as *a key together with the identifying relationship*

Representation of Relationship Types

❑ How to Represent a Relationship Type and its Constraints?

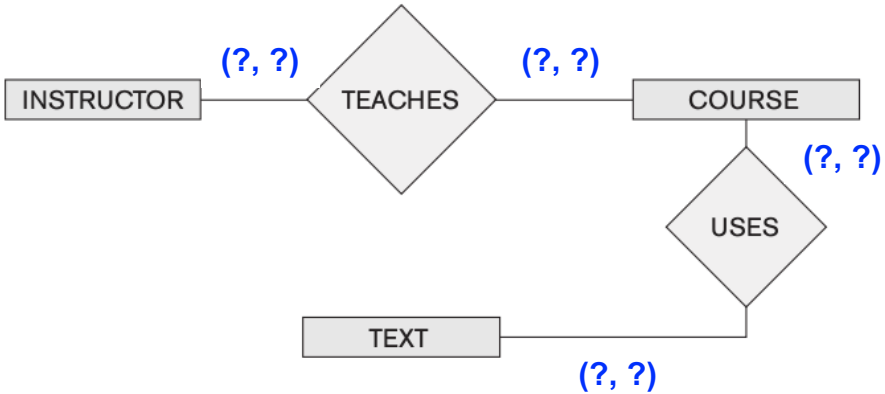






Exercise 1: Constraints on ER Diagram

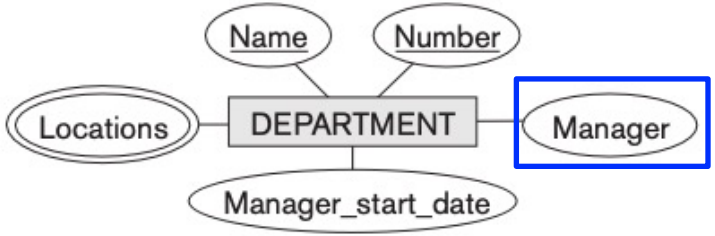
- **Suggest the (min, max) constraints on the following diagram**
  - A course may or may not use a textbook
  - A course may not use more than five books
  - A course can be taught by at least one instructor and by a maximum of 4 instructors
  - A textbook is used in some courses
  - An instructor teaches from two to four courses

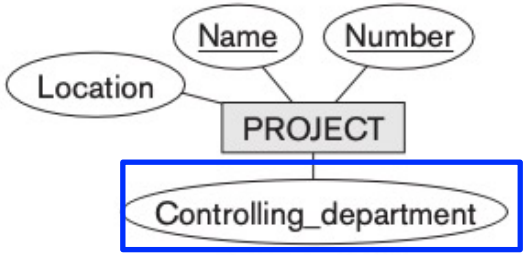


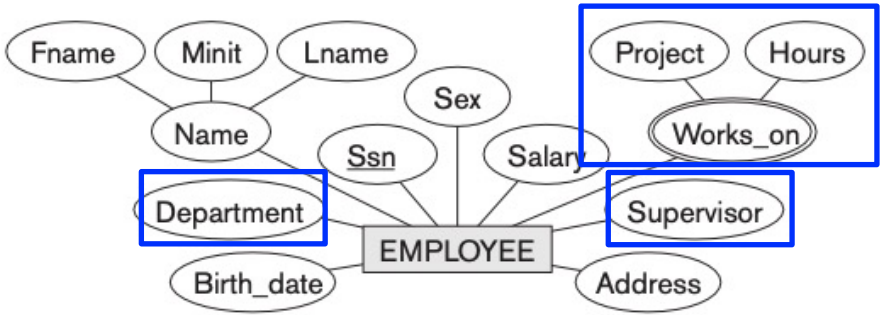
Refining Conceptual Design

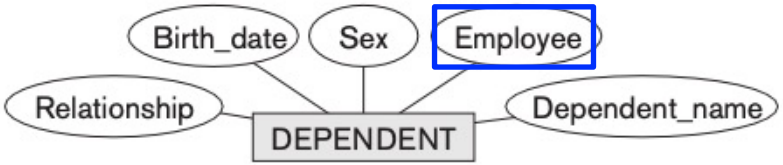
❑ Let's Refine the Conceptual Design for COMPANY Database

- By changing the attributes representing relationships into **relationship types**





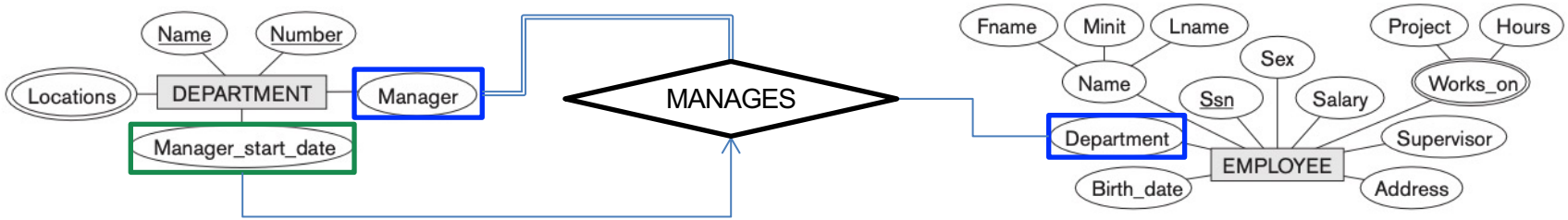




Refining Conceptual Design: MANAGES

❑ Relationship Type 1: MANAGES

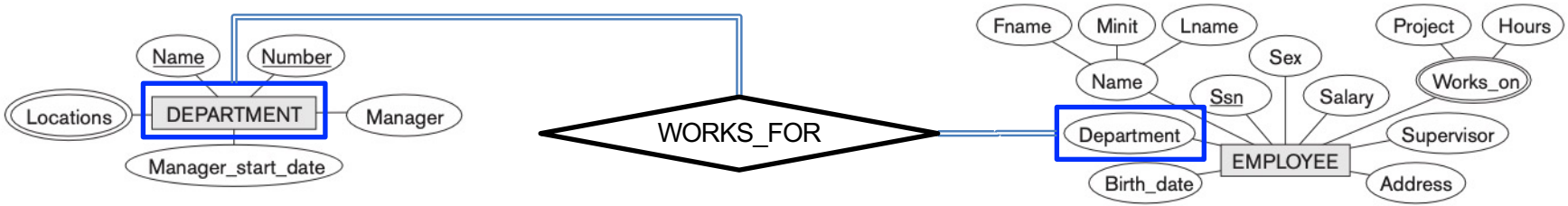
- Participating entity types: **EMPLOYEE** and **DEPARTMENT**
- Cardinality ratio: **1:1**
- EMPLOYEE participation is **partial**, DEPARTMENT participation is **total**
- The attribute **Manager_start_date** is assigned to the relationship type
  - *This attribute can be assigned to one of the DEPARTMENT and EMPLOYEE entity type



Refining Conceptual Design: WORKS_FOR

❑ Relationship Type 2: WORKS_FOR

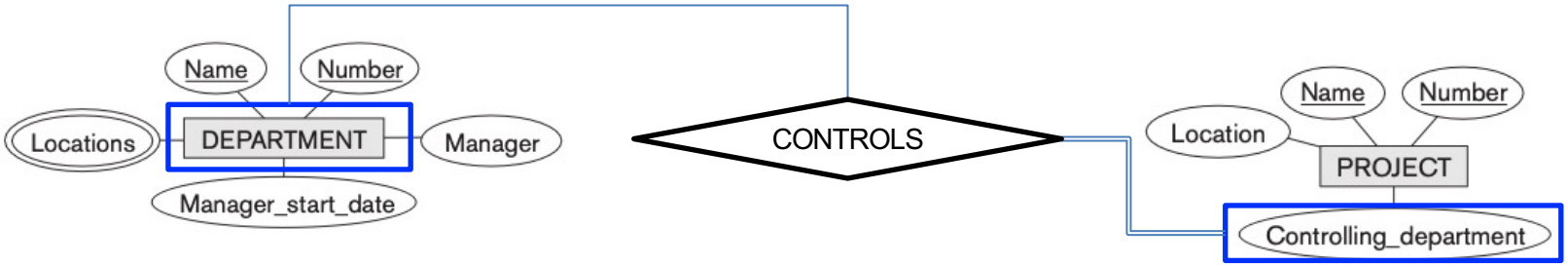
- Participating entity types: **EMPLOYEE** and **DEPARTMENT**
- Cardinality ratio: **1:N**
  - i.e., 1-side: DEPARTMENT, N-side: EMPLOYEE
- Both participations are **total**
- No relationship type attribute



Refining Conceptual Design: CONTROLS

❑ Relationship Type 3: CONTROLS

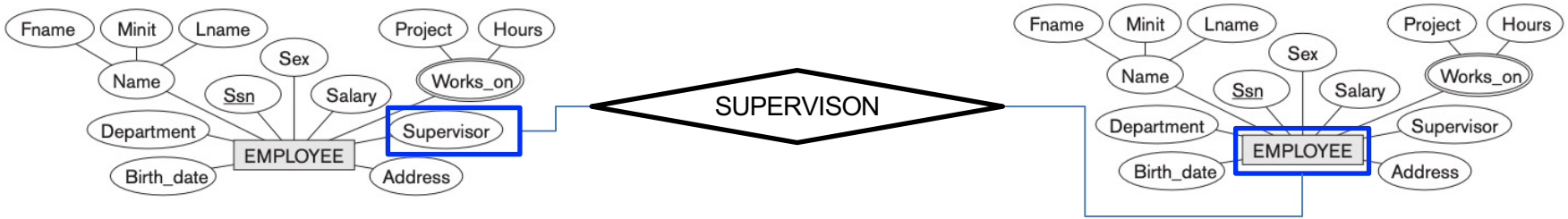
- Participating entity types: **DEPARTMENT** and **PROJECT**
- Cardinality ratio: **1:N**
  - ❑ i.e., 1-side: DEPARTMENT, N-side: PROJECT
- PROJECT participation is **total**, DEPARTMENT participation is **partial**
- No relationship type attribute



Refining Conceptual Design: SUPERVISION

❑ Relationship Type 4: SUPERVISION

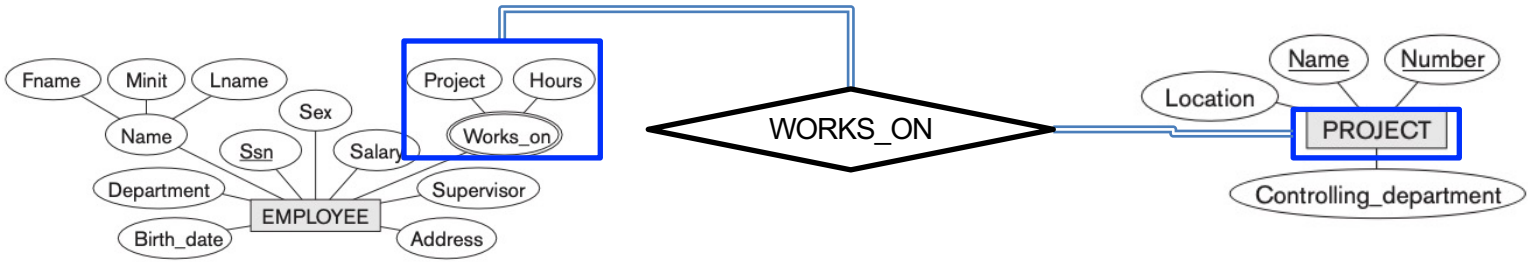
- Participating entity types: **EMPLOYEE**
- Cardinality ratio: **1:N**
  - ❑ i.e., 1-side: EMPLOYEE (in the supervisee role), N-side: EMPLOYEE (in the supervisor role)
- Both participations are **partial**
- No relationship type attribute



Refining Conceptual Design: WORKS_ON

❑ Relationship Type 5: WORKS_ON

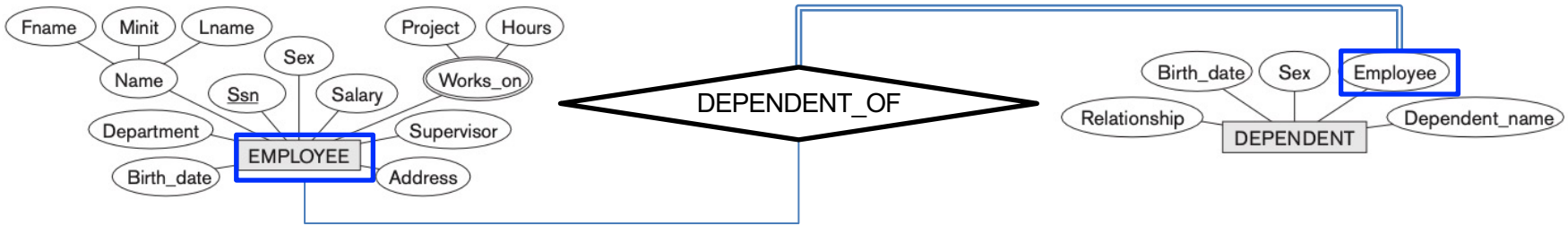
- Participating entity types: **EMPLOYEE** and **PROJECT**
- Cardinality ratio: **M:N**
  - i.e., an employee can work on several projects and a project can have several employees
- Both participations are **total**
  - An employee must work on at least one project; a project must be conducted by at least one employee
- The attribute **Hours** is assigned to the relationship type
  - It is determined by an employee-project combination



Refining Conceptual Design: DEPENDENT_OF

❑ Relationship Type 6: DEPENDENTS_OF

- Participating entity types: **EMPLOYEE** and **DEPENDENT**
  - **Identifying entity type**: EMPLOYEE, **weak entity type**: DEPENDENT
  - The DEPENDENTS_OF is the identifying relationship for the weak entity type DEPENDENT
- Cardinality ratio: **1:N**
  - 1-side: EMPLOYEE, N-side: DEPENDENT (i.e., an employee can have several dependents)
- EMPLOYEE participation is **partial**, DEPENDENT participation is **total**
  - i.e., an employee may or may not have dependents, but a dependent must have its employee



Refining Conceptual Design: Result

❑ Process of Refinement

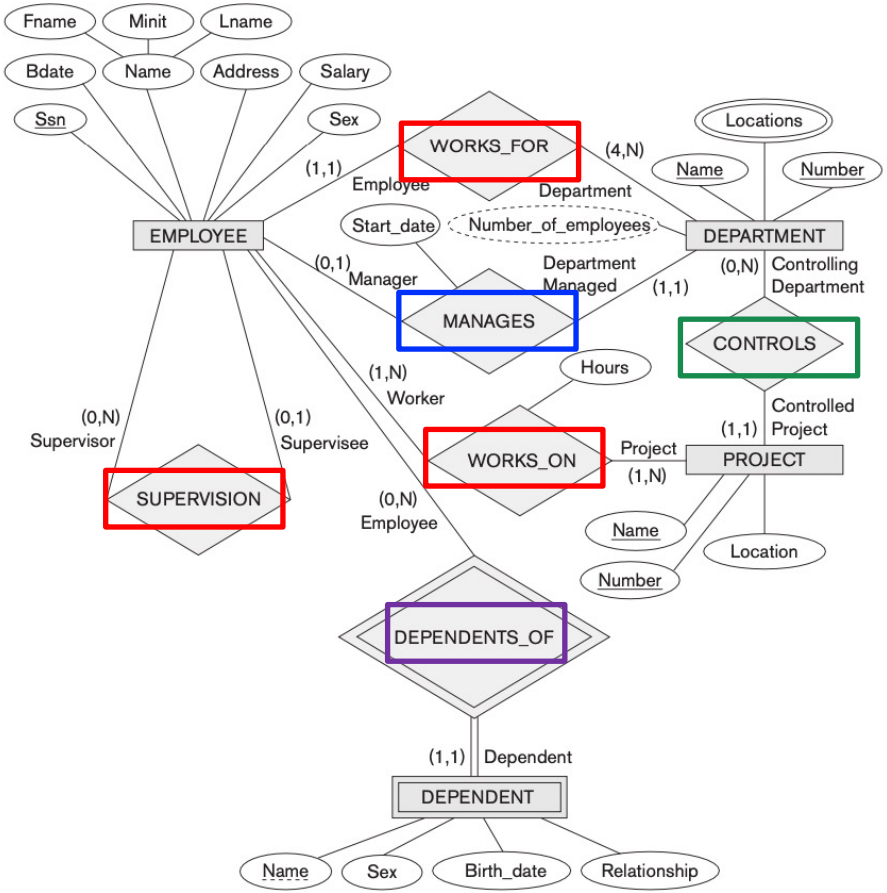
- Specifying all relationship types
- Changing the attributes representing relationships into **relationship types**

Manager and Manager_start_date from DEPARTMENT
→ To MANAGES

Controlling_department from PROJECT
→ To CONTROLS

Department, Supervisor, and Works_on from EMPLOYEE
→ To WORKS_FOR, EMPLOYEE, and WORKS_ON

Employee from DEPENDENT
→ To DEPENDENTS_OF

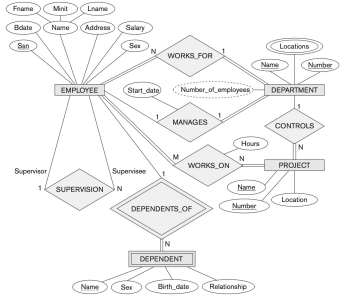


Takeaways: Database System Architecture

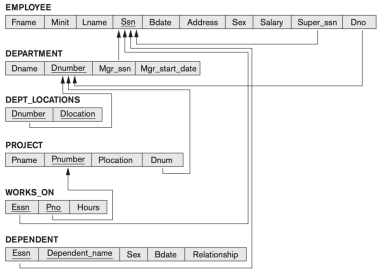
❑ Data Model: A Collection of Concepts Describing DB Structure



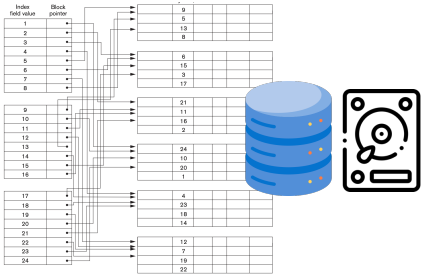
Real-world Data



Conceptual Data Model



Representational Data Model



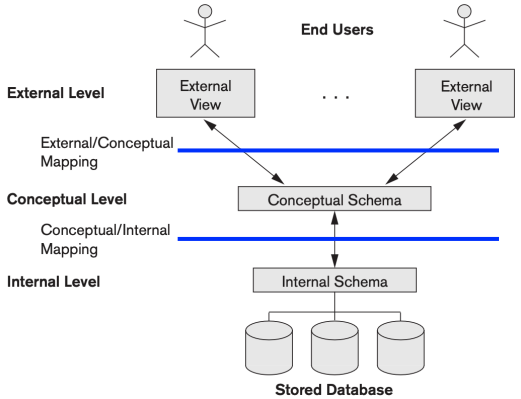
Physical Data Model

❑ Three-Schema Architecture

- Internal, conceptual, and external schemas

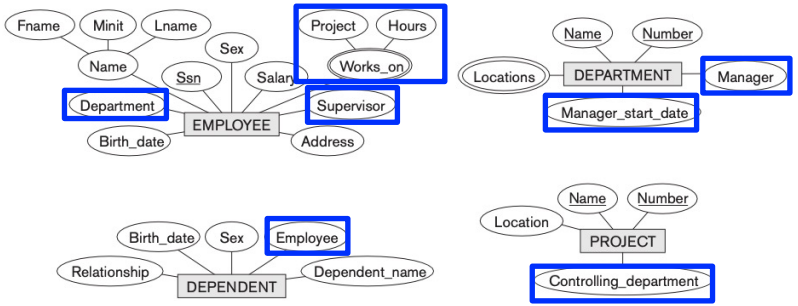
❑ Data Independence

- Logical data independence
- Physical data independence

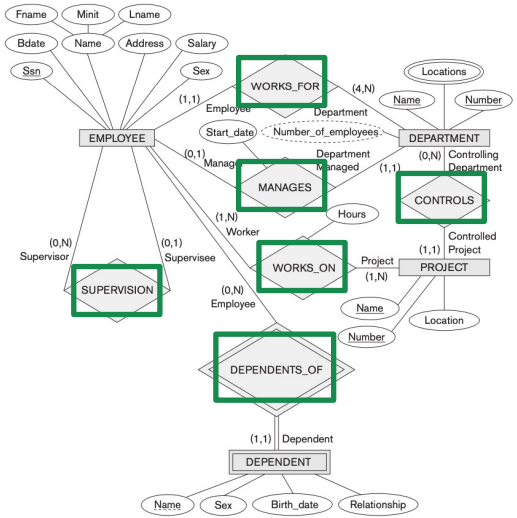


Takeaways: ER Data Modeling

- **Process of Designing ER Schema Diagram**
  - (1) Describe entity types and their attributes based on the data requirements
  - (2) **Specify relationship types and their constraints**
  - (3) **Change the attributes representing relationships into relationship types**
- **Focus on the Schema, Rather Than the Instances**







Course Schedule (Tentative)

Week		Topic	Week		Topic
1	~~2026.09.01~~	~~Course Introduction~~	9	2026.10.27	AI for Databases: NL2SQL
	~~2026.09.04~~	~~No class~~		2026.10.30	Hands-on Practice
2	2026.09.08	**ER Model (Video Lecture)**	10	2026.11.03	AI for Databases: NL2SQL
	2026.09.11	Hands-on Practice		2026.11.06	Hands-on Practice
3	2026.09.15	Relational Database (Video Lecture)	11	2026.11.10	Disk Storage and File Structure
	2026.09.18	ER-to-Relational Mapping (Video Lecture)		2026.11.13	Parallelizing Disk Access
4	2026.09.22	Basic SQL (Video Lecture)	12	2026.11.17	Indexing for Physical Database Design (Assignment #2)
	2026.09.25	No class (추석연휴)		2026.11.20	Advanced Indexing Techniques
5	2026.09.29	Advanced SQL (Assignment #1)	13	2026.11.24	Query Processing
	2026.10.02	Hands-on Practice		2026.11.27	Query Optimization
6	2026.10.06	Relational Algebra	14	2026.12.01	Transaction Processing
	2026.10.09	Hands-on Practice (Video Lecture, 한글날)		2026.12.04	Concurrency Control and Recovery
7	2026.10.13	Functional Dependencies and Normalization	15	2026.12.08	Advanced Topic: TBD
	2026.10.16	Hands-on Practice		2026.12.11	No class (Q&A)
8	2026.10.23	**Midterm Exam**	16	2026.12.18	**Final Exam**

Q&A:
DB System Architecture & ER Model